Project Name - DegreePath

Team Members - Omkar Gadute, Achyuthan Sivasankar, Maitri Savaliya, Han Yang, Hanyue Liao

Group - Software Engineering I I2 - 1

Instructor - Raz Sarem

Date - 20 October 2026

Course Name - Software Engineering I

FALL 26 - CS GY 6063 - 1 I2

AI disclosure - GPT5 used for proofreading; and ADR-03, reviewed and edited by the team

2 Architecture Overview

2.1 Architectural style and responsibilities

DegreePath uses a client-server architecture with a React and TypeScript frontend, a modular Django backend, and PostgreSQL. React renders student and catalog-administrator workflows in the browser and calls Django's JSON API over HTTPS.

Django owns authentication, authorization, profile and catalog operations, planning, validation, saved plans, and CSV export. The six backend modules have separate responsibilities and share one deployment.

Users sign in through Django. A 24-hour session cookie identifies later requests; React sends a CSRF token with requests that change data. Django checks the user's role and, for profiles and plans, ownership before acting. Catalog imports validate files and degree rules before committing a complete update. The planner reads a consistent profile, catalog, and requirement snapshot, runs OR-Tools CP-SAT, and sends the result to the plan validator. Saved plans are checked again on save; their freshness is determined when read from profile and catalog versions.

2.2 Component view

Figure 1 shows the React client, six Django application modules, and PostgreSQL. Django's API routes handle session and CSRF checks before calling the modules. The Catalog module uses NetworkX for rule checks; the Planner uses OR-Tools CP-SAT. Modules that persist data use Django's ORM to access PostgreSQL.

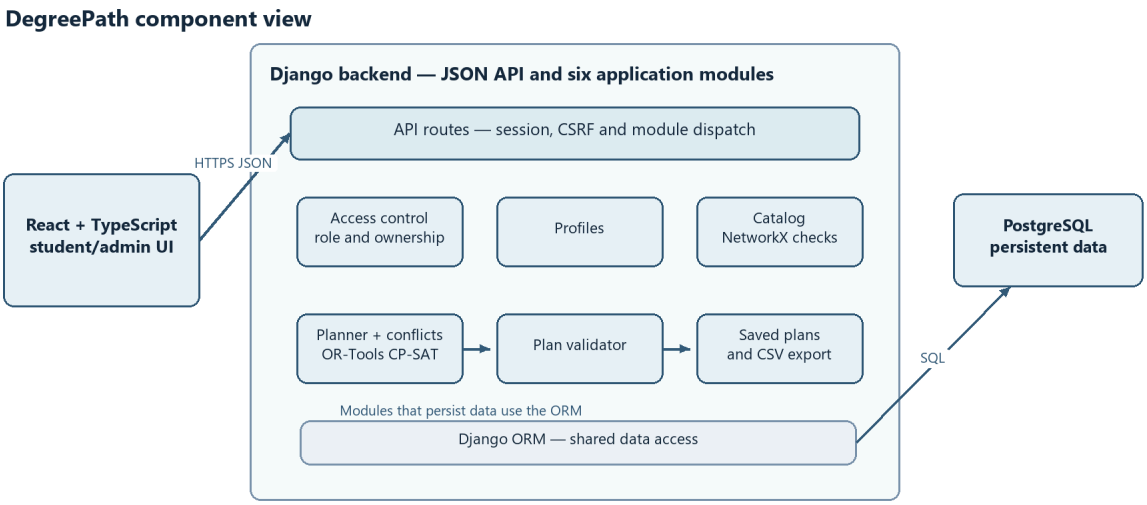


Figure 1. DegreePath component view.

2.3 Deployment view

The hosted demo serves the compiled React files and Django JSON API from one web origin. The browser loads React over HTTPS and sends API requests with its Django session cookie; requests that change data also carry a CSRF token. Django connects to managed PostgreSQL through a private database link. Local development and CI use the same schema with separate test data.



Figure 2. DegreePath deployment view.

2.4 Fit to the MVP and non-functional requirements

Server-side role and ownership checks address NFR-1. Argon2 password hashes and 24-hour Django sessions address NFR-2. Transactions make catalog and requirement imports atomic for NFR-4, while PostgreSQL persistence supports NFR-6. Planner time limits and a separate plan validator support the NFR-3 and NFR-4 checks against the Sprint 1 fixtures.

3 Modeling Choices and UML Decision

Component view: Figure 1 uses box-and-arrow notation to show the React client, six Django modules, and PostgreSQL. Section 5 specifies the module interfaces, so UML interface symbols would repeat that detail.

Deployment view: Figure 2 uses a simple box-and-arrow layout for the browser, the hosted Django application serving React files, and PostgreSQL. These three locations and their connections are clear without UML node notation.

Domain model**:** Figure 3 uses a Crow's-foot Entity-Relationship Diagram (ERD) rather than a UML class diagram. Because DegreePath's domain entities map directly to PostgreSQL tables via Django's ORM and hold no complex inheritance hierarchies or polymorphic methods, Crow's-foot notation more clearly communicates primary keys (PK), foreign keys (FK), unique constraints (UK), and exact join-table cardinalities alongside the attribute constraints in Section 4.2.

4 Data Design

4.1 Domain model

The domain model covers every entity the MVP stories (US-01 to US-13 and US-18) need. It has four groups:

- **Accounts:** User, StudentProfile, CompletedCourse
- **Catalog:** Course, CourseOffering, PrereqClause, PrereqClauseCourse, Corequisite
- **Degree rules:** DegreeProgram, Requirement, RequirementCourse
- **Plans and imports:** Plan, PlanEntry, CatalogImport, CatalogState

Figure 3 (next page) shows the persistent entities with their key attributes, primary keys (PK), foreign keys (FK) and unique keys (UK). Crow's-foot ends show cardinality: a single bar means one, a crow's foot means many, and a circle means optional. Colours group the entities: blue for accounts, green for the catalog, yellow for degree rules, orange for plans, and grey for import bookkeeping.

Domain concepts that are not stored in tables:

- **Login session.** Sessions are managed by the framework's session store and expire after 24 hours (NFR-2). They are tied to User and hold no planning data.
- **Conflict set (US-12).** This is a transient result of a failed generation. It is a closed list of ConstraintRef objects, each of one of nine types defined in Section

5.4: CREDIT_CAP (StudentProfile.credit_limit), TARGET_TERM (the profile's target term), PREREQ (PrereqClause), COREQ (Corequisite), OFFERING (CourseOffering), REQUIRED_COURSE, POOL and CREDIT_THRESHOLD (Requirement, by its type), and MANUAL_REVIEW (a Course with prereq_manual_review = true). The interface schema and the US-12 tests both use this list. A course flagged for manual review is never scheduled unless it is already completed (US-06 AC3), so if a required course depends on one, generation is infeasible and the conflict set names that course as MANUAL_REVIEW. The result also carries a "possibly non-minimal" flag. It is recomputed on demand, so no table is needed.

- **Import report.** The structured errors and manual-review flags are stored in CatalogImport.report.

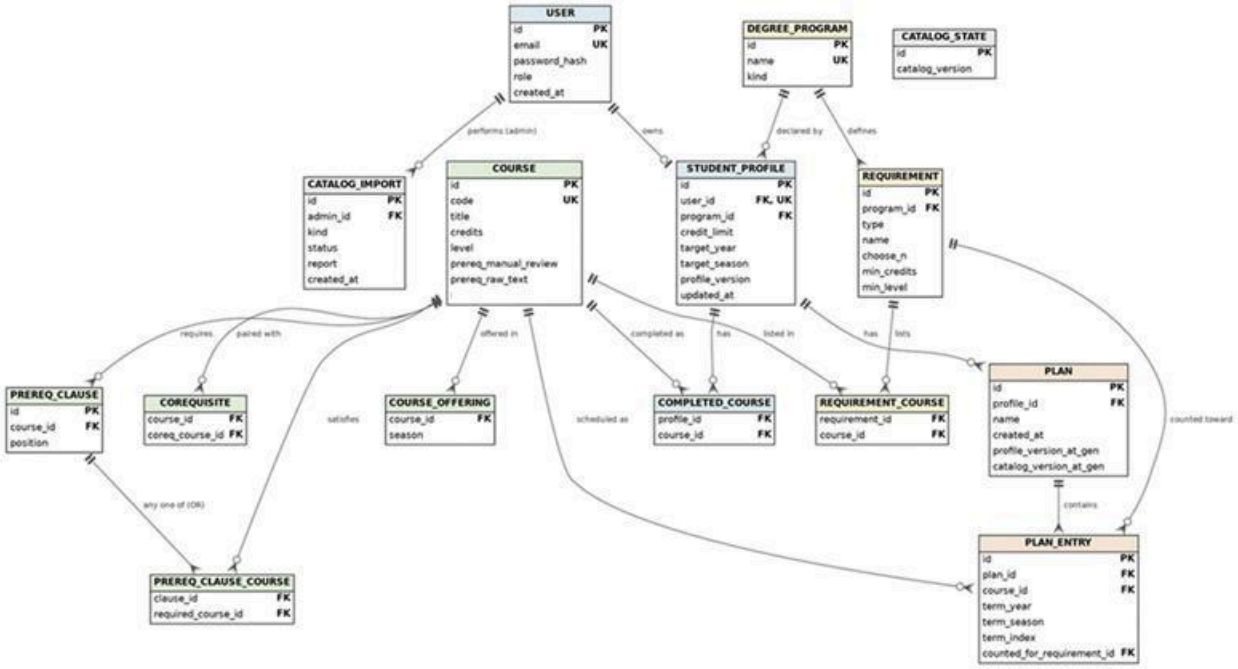


Figure 3. DegreePath domain model (entity-relationship diagram).

4.2 Entities, attributes and constraints

"Req." means the field is required (NOT NULL). Unique constraints are enforced by the database.

Accounts and profiles

Entity	Attributes (Req. = NOT NULL)	Uniqueness	Other constraints

User	id, email, password_hash, role, created_at (all Req.)	email	Argon2 hash only, no plaintext. role ∈ {student, admin}. A password of 8+ characters is checked before hashing (US-01 AC1).
StudentProfile	id, user_id, profile_version, updated_at (Req.). program_id, credit_limit, target_year, target_season (nullable until the student sets them)	id; user_id (1:1 with User)	id is a random UUID, so profile IDs cannot be guessed (Section 5.1). credit_limit is an integer from 1 to 24 (CHECK). Target term not in the past (service-layer check). Generation is blocked if program, credit limit or target term is null (US-09 AC3; the program is included because the degree requirements come from it).
CompletedCourse	profile_id, course_id (Req.)	(profile_id, course_id)	FK to Course, so unknown codes are rejected (US-02 AC2).

Catalog

Entity	Attributes (Req. = NOT NULL)	Uniqueness	Other constraints
Course	id, code, title, credits, level, prereq_manual_review (Req.). prereq_raw_text (nullable)	code	credits > 0 (CHECK, US-05 AC2). level is derived from the course number. Re-importing an existing code updates it and creates no duplicate (US-05 AC3).
CourseOffering	course_id, season (Req.)	(course_id, season)	season ∈ {Fall, Spring}. Anything else rejects the import (US-05 AC5).
PrereqClause	id, course_id, position (Req.)	(course_id, position)	One row per AND-ed group. A clause with one course is mandatory; a clause with several courses is an OR group (Section 4.5, rule 3).
PrereqClauseCourse	clause_id, required_course_id (Req.)	(clause_id, required_course_id)	A clause must have at least one course. An unknown code rejects the import (US-06 AC4). A course cannot require itself.

Corequisite	course_id, coreq_course_id (Req.)	(course_id, coreq_course_id)	CHECK course_id < coreq_course_id, so each pair is stored once. Stored separately from prerequisites (US-06 AC2, US-07 AC3).

Degree rules

Entity	Attributes (Req. = NOT NULL)	Uniqueness	Other constraints
DegreeProgram	id, name, kind (Req.)	name	kind ∈ {major, minor}. The MVP loads one pilot major.
Requirement	id, program_id, type, name (Req.). choose_n, min_credits, min_level (conditional)	(program_id, name)	type ∈ {required, pool, credit_threshold}. A pool needs choose_n ≥ 1 and ≤ pool size (US-08 AC2). A threshold needs min_credits > 0.
RequirementCourse	requirement_id, course_id (Req.)	(requirement_id, course_id)	FK to Course, so unknown courses are rejected (US-08 AC3).

Plans and imports

Entity	Attributes (Req. = NOT NULL)	Uniqueness	Other constraints
Plan	id, profile_id, name, created_at, profile_version_at_gen, catalog_version_at_gen (all Req.)	id	Owned by exactly one student (profile_id is a FK to StudentProfile). Delete cascades to PlanEntry (US-11). Only valid plans are saved: infeasible results are never stored, and on save the server reloads the snapshot and re-runs the Plan validator, rejecting any plan with violations, so a plan edited in the browser cannot be saved (NFR-4; Section 5.3).
PlanEntry	id, plan_id, course_id, term_year, term_season, term_index (Req.). counted_for_requirement_id (nullable)	(plan_id, course_id)	A course appears at most once per plan. term_index orders terms chronologically for display and CSV (US-13 AC1). A course in two pools counts toward only one (US-18 AC2).
CatalogImport	id, admin_id, kind, status, created_at, report (Req.)	id	status ∈ {accepted, rejected}. kind ∈ {courses, requirements}. Prerequisites

			and corequisites arrive with the course file.
CatalogState	id, catalog_version (Req.)	single row	Incremented only when an import is accepted.

4.3 Relationships

Relationship	Cardinality	Notes
User – StudentProfile	1 : 0..1	Admins have no profile.
StudentProfile – CompletedCourse – Course	M : N	Via the CompletedCourse join table.
StudentProfile – Plan	1 : N	A student may save many plans (Plan.profile_id).
StudentProfile – DegreeProgram	N : 1	The declared major.
Course – PrereqClause	1 : N	Clauses are AND-ed together.
PrereqClause – Course (via PrereqClauseCourse)	M : N	Courses within a clause are OR-ed.
Course – Course (via Corequisite)	M : N	Self-referencing, stored once per pair.
Course – CourseOffering	1 : N	One row per season offered.
DegreeProgram – Requirement	1 : N	A program defines its requirements.
Requirement – Course (via RequirementCourse)	M : N	Required courses and pool members.
Plan – PlanEntry	1 : N	Entries are owned by the plan.
PlanEntry – Requirement	N : 0..1	Which pool a course counts toward.
User (admin) – CatalogImport	1 : N	Audit trail of uploads.

4.4 Valid states

Item	Valid states	Transitions and rules
User.role	student, admin	Fixed at account creation. Students register themselves. The admin is seeded.
CatalogImport.status	accepted, rejected	A rejected import changes nothing in the catalog (US-07 AC2). An accepted one commits atomically and increments catalog_version.
Plan freshness (derived, not stored)	current, out of date	A plan is out of date when its profile_version_at_gen or catalog_version_at_gen differs from the current value (US-03 AC2). A stale plan cannot be exported as current.

Plan existence	saved, deleted	Only valid plans are saved: the server re-runs the Plan validator on save and rejects any plan with violations (NFR-4). Deletion is permanent, and the plan URL then returns 404 (US-11 AC1).
Generation result (transient)	valid plan, no valid plan (with conflict set)	An infeasible result is never stored. If the solver hits its time limit, the conflict set is labeled “possibly non-minimal” (US-12 AC4).
Course prerequisite handling	structured, manual review	prereq_manual_review = true means the prose rule is not treated as satisfied (US-06 AC3). Such a course is never scheduled unless already completed. If a required course depends on one, generation is infeasible and the conflict set names it as MANUAL_REVIEW (Section 5.4).
Profile completeness	incomplete, ready	A profile is ready when program, credit limit and target term are all set. Generation is blocked otherwise, and the error lists the missing fields (US-09 AC3).

4.5 Import-time integrity rules

These checks keep bad data out of the planner. They run in one transaction, so any failure rolls everything back (NFR-4):

1. Every row has valid credits and valid term-offering values, or the error names the row and column.
2. Every prerequisite, corequisite and requirement refers to a course in the catalog.
3. The mandatory prerequisite graph has no cycle, and the error shows the path (for example “A → B → C → A”). Nodes are courses. An edge A → B is added only for a clause that lists a single course (A requires B, with no alternative). Clauses with two or more courses (OR groups) add no edges, because another course in the group can still satisfy them; those cases are left to the solver. Cycle detection runs on this graph with NetworkX.
4. A corequisite pair is accepted unless a prerequisite chain forces the two courses into different terms, which is rejected as a loop (US-07 AC4). The check is: (a) merge each corequisite pair into one node (pairs that share a course merge into one node); (b) redraw the mandatory prerequisite edges from rule 3 between the merged nodes; (c) run the same cycle detection. Any cycle is rejected, including an edge from a merged node to itself (A requires B while A and B are corequisites). The error uses the original course codes, for example “A → B → C ⇄ A”.
5. Each pool has choose_n no larger than its number of courses.

4.6 Story coverage check

Story	Entities used
US-01 Sign-in	User, session
US-02 / US-03 Profile	StudentProfile, CompletedCourse, DegreeProgram
US-04 Catalog search	Course, CourseOffering, PrereqClause(+Course), Corequisite
US-05 Course import	Course, CourseOffering, CatalogImport, CatalogState
US-06 / US-07 Rules and cycles	PrereqClause(+Course), Corequisite, CatalogImport
US-08 Requirements	DegreeProgram, Requirement, RequirementCourse
US-09 Generate	All catalog, rule and profile entities (read-only), producing a Plan
US-10 / US-11 Save and delete	Plan, PlanEntry
US-12 Conflicts	Conflict set (transient) over StudentProfile, PrereqClause, Corequisite, CourseOffering, Requirement, Course
US-13 CSV export	Plan, PlanEntry, Course
US-18 Major rules	Requirement, RequirementCourse, PlanEntry

5 Interface Design

DegreePath exposes one JSON API under `/api/`. The React app calls it for every data operation, so access rules are enforced in one place. Section 5.3 lists every endpoint with its caller, inputs, success output, errors and the stories it supports. Section 5.5 lists the internal module interfaces behind them. The module names (Access control, Profiles, Catalog, Planner, Plan validator, Saved plans) are the same ones used in Figure 1, the lifelines of Figures 4 to 6, and Section 5.5.

5.1 Conventions

- **Format.** Request and response bodies are JSON, except catalog and requirement uploads (multipart file) and CSV export (`text/csv`).
- **Terms.** A term is `{"season": "Fall" | "Spring", "year": 2027}` and is shown to users as "Fall 2027".
- **Sessions.** Django session cookie (HttpOnly, Secure, SameSite=Lax) that expires 24 hours after sign-in (NFR-2). Requests that change data must carry Django's CSRF token.
- **Pages and API.** The React app calls the API for all data; on a 401 response it routes the user to the sign-in page (US-01 AC4, US-04 AC3 and AC4).
- **IDs.** `StudentProfile.id` and `Plan.id` are random UUIDs, so record IDs cannot be guessed by counting.

- **Errors.** Every error body has one shape: `{"error": {"code", "message", "details"}}`. `message` is the text the UI shows; `details` lists field-level or row-level problems.
- **Logging.** Bodies of profile and plan requests are never logged. Errors log the error code and record IDs only (NFR-5).

Status	Meaning in DegreePath	Example
200, 201, 204	Success.	Plan generated, import accepted, plan deleted
400	The request itself is invalid: a field fails validation, a course code is unknown, or an uploaded file breaks an import rule.	Credit limit of 25 (US-03 AC4)
401	Not signed in: no session, an expired session, or a failed sign-in.	Signed-out catalog request (US-04 AC3)
403	Signed in, but the role or ownership check fails.	Student opens another student's plan (US-10 AC3)
404	The resource does not exist or was deleted.	URL of a deleted plan (US-11 AC1)
409	The request is valid but conflicts with stored state.	Email already registered (US-01 AC3); profile incomplete (US-09 AC3); plan out of date
500	Unexpected server error. The response never contains a partial result.	Validator rejects solver output (Figure 5)
504	The planner hit its time cap with neither a plan nor proof that none exists.	NFR-3

5.2 Access rules

Every API view calls Access control before any other work. The checks run in this order, and the first one that fails decides the response.

1. Session present and less than 24 hours old; otherwise **401**.
2. Role allowed for the endpoint (student or admin); otherwise **403**.
3. Resource exists; otherwise **404**.
4. Caller owns the resource (the profile or plan belongs to the caller); otherwise **403**.

Because existence is checked before ownership, a student can tell whether a guessed plan ID exists. The team accepts this: US-10 AC3 and US-11 AC3 require 403 for another student's plan, and UUIDs make guessing impractical. Callers in the tables below are **Public** (no session), **Signed-in** (any role), **Student**, **Owner** (the student who owns the profile or plan) and **Admin**.

5.3 API endpoints

Accounts (Access control)

Endpoint and caller	Input	Success	Errors	Stories
POST /api/auth/register Public	email; password (8+ characters)	201 {user_id, role: "student", profile_id}. Creates the User and an empty StudentProfile, then starts a session.	400 INVALID_EMAIL; 400 PASSWORD_TOO_SHORT; 409 EMAIL_TAKEN ("Email already registered")	US-01 AC1, AC3; NFR-2
POST /api/auth/login Public	email; password	200 {user_id, role, profile_id for students}. Sets the session cookie.	401 INVALID_CREDENTIALS ("Incorrect email or password", the same whether or not the email exists). No session is created.	US-01 AC2; NFR-2
POST /api/auth/logout Signed-in	None	204. Session deleted.	401	US-01
GET /api/auth/csrf Public	None	204. Sets the csrftoken cookie; the React app sends it back in the X-CSRFToken header on every request that changes data.	None	US-01; NFR-2

Profiles

Endpoint and caller	Input	Success	Errors	Stories
GET /api/profiles/{id} Owner	None	200 {id, program, completed_courses, credit_limit, target_term, profile_version, updated_at, ready}	401; 403 NOT_OWNER; 404	US-02 AC1; US-03 AC1; NFR-1
PUT /api/profiles/{id} Owner	program_id; completed_courses (codes); credit_limit (1–24 or null); target_term (or null). Full replacement, saved all-or-nothing.	200 updated profile. updated_at is set; profile_version goes up by 1 if any of the four fields changed.	400 VALIDATION_ERROR with field details (credit limit outside 1–24, target term in the past, unknown program); 400 UNKNOWN_COURSE naming each code; 401; 403 NOT_OWNER; 404. On any error the profile is unchanged.	US-02 AC1–AC3; US-03 AC1–AC4; NFR-1

Catalog and degree rules (Catalog)

Endpoint and caller	Input	Success	Errors	Stories

GET /api/courses?q={text} Signed-in	q: code or title fragment, case-insensitive. At most 50 results.	200 {results: [{code, title, credits, prerequisites as "A AND (B OR C)", corequisites, offered, manual_review}]}. Empty list when nothing matches.	401	US-04 AC1–AC3
GET /api/programs **GET /api/programs/{id}** Signed-in	None	200 list of {id, name}; for one program, its requirement summary (required courses, pools as "choose N of M", credit thresholds).	401; 404	US-02; US-08 AC1
POST /api/admin/catalog-imports Admin	CSV or JSON file: code, title, credits, offered seasons, prerequisite expression, corequisites	201 {import_id, imported, created, updated, manual_review (codes), catalog_version}. UI shows "N courses imported".	400 IMPORT_REJECTED {import_id, errors: [{row, column, value, message}]} for missing or non-numeric credits, invalid term, unknown course code, prerequisite cycle ("A → B → C → A") or corequisite contradiction ("A → B → C ⇄ A"); 400 UNSUPPORTED_FILE; 500 IMPORT_FAILED; 401; 403. The catalog is unchanged after any error.	US-05 AC1–AC5; US-06 AC1–AC4; US-07 AC1–AC4; NFR-1; NFR-4
POST /api/admin/requirement-imports Admin	JSON file: program name, then requirements of type required (courses), pool (courses, choose_n) or credit_threshold (min_credits, min_level)	201 {import_id, program_id, summary, catalog_version}. Replaces that program's previous definition in one transaction.	400 IMPORT_REJECTED naming the pool whose choose_n exceeds its size, or the unknown course; 401; 403. The previous definition is unchanged after any error.	US-08 AC1–AC4; NFR-1; NFR-4

Plans (Planner, Plan validator, Saved plans)

Endpoint and caller	Input	Success	Errors	Stories
POST /api/plans/generate Student	None. Always uses the caller's own stored profile, so another student's data cannot be targeted.	200 {status: "valid", terms: [{term, credits, courses: [{code, title, credits, counted_for}]}], profile_version, catalog_version} or 200 {status: "infeasible", message: "No valid plan exists for these constraints",	409 PROFILE_INCOMPLETE {missing}; 504 PLANNER_TIMEOUT; 500 INTERNAL_ERROR; 401; 403 for admins	US-09 AC1–AC5; US-12 AC1–AC4; US-18 AC1–AC3; NFR-3

		conflicts (Section 5.4), possibly_non_minimal}		
POST /api/plans Student	name; entries [{course_code, term, counted_for}]; profile_version and catalog_version from the generate response	201 {id, name, created_at}. The server reloads the snapshot and re-runs the Plan validator before saving; the browser's copy is never trusted. The validator also checks the pool allocation: each counted_for course belongs to that pool, no course counts twice, and every pool reaches choose_n.	400 PLAN_INVALID {violations}; 409 STALE_PLAN (profile or catalog changed since generation); 400 VALIDATION_ERROR (name); 401; 403	US-10 AC1; NFR-4
GET /api/plans Student	None	200 {plans: [{id, name, created_at, is_current}]}. Empty list when there are none; the UI shows "No plans yet".	401; 403 for admins	US-10 AC2; US-03 AC2
GET /api/plans/{id} Owner	None	200 full plan with counted_for on each course, and is_current. When is_current is false the UI shows "Out of date – regenerate".	401; 403 NOT_OWNER; 404	US-10 AC1, AC3; US-03 AC2; US-18 AC2; NFR-1
DELETE /api/plans/{id} Owner	None. The confirm dialog is in the browser; Cancel sends no request.	204. Plan and PlanEntry rows are removed permanently; later requests return 404.	401; 403 NOT_OWNER; 404	US-11 AC1–AC3; NFR-1
GET /api/plans/{id}/export.csv Owner	None	200 text/csv with header term,course_code,title,credits and one row per course, ordered by term_index then course_code.	409 PLAN_OUT_OF_DATE; 401; 403 NOT_OWNER; 404	US-13 AC1–AC4; NFR-1

US-13 AC3 (Export disabled until a valid plan exists) holds by design: infeasible results are never saved, so the export endpoint can only ever return a saved, valid plan.

5.4 Conflict set format

A conflict set is a list of ConstraintRef objects, each `{type, ref, text}`. `type` is one of the nine values below, `ref` points to the row the student would change, and `text` is the line shown on screen. Only these types can appear, so the US-12 tests can assert on them directly.

Type	Refers to	Example text shown to the student
CREDIT_CAP	StudentProfile.credit_limit	"At most 12 credits per term"
TARGET_TERM	StudentProfile target term	"Graduate by Spring 2028 (4 terms left)"

PREREQ	PrereqClause	"CS-301 needs one of CS-201, CS-202 in an earlier term"
COREQ	Corequisite	"CS-210 and CS-211 must be taken in the same term"
OFFERING	CourseOffering	"CS-340 is offered only in Fall"
REQUIRED_COURSE	Requirement (type required) and the course	"Core requirement: CS-401"
POOL	Requirement (type pool)	"Systems electives: choose 3 of 6"
CREDIT_THRESHOLD	Requirement (type credit_threshold)	"At least 12 credits at 3000-level or above"
MANUAL_REVIEW	Course with prereq_manual_review = true	"CS-499 needs instructor permission, which DegreePath cannot check"

Three rules are hard constraints and never appear in a conflict set, because a student cannot change them: a course is scheduled at most once; completed courses are not rescheduled (US-09 AC2); and no course is scheduled before the next term. A course flagged for manual review is never scheduled unless it is already completed (US-06 AC3). This is an explainable constraint, not a hard one: if a requirement can only be met through such a course, generation is infeasible and the conflict set names the course with MANUAL_REVIEW, so the student knows to get permission outside DegreePath (instructor-permission prerequisites are out of scope from Sprint 1).

5.5 Module interfaces

A **snapshot** is an immutable PlanningInput: the profile fields, the completed-course set, the catalog and requirement rows, and the two version numbers, all read in one transaction (Figure 4). The Plan validator never calls the solver or the database. Keeping it independent means a bug in the CP-SAT model shows up as a validator failure in tests, not as a wrong plan on screen.

Module	**Operation**	**Returns or raises**	**Used by and stories**
Access control	authenticate(request)	User, or NotAuthenticated (401)	Every API view; US-01; NFR-2
Access control	require_role(user, role) require_owner(user, obj)	Nothing, or Forbidden (403)	Every API view; NFR-1
Profiles	get_profile(user, id) update_profile(user, id, data)	StudentProfile, or ValidationError / UnknownCourses (400)	US-02; US-03
Catalog	search_courses(q, limit=50)	List of CourseView	US-04
Catalog	run_import(file, kind, admin)	ImportReport(status, counts, errors, manual_review, catalog_version). Bad data returns a rejected report; there is never a partial write.	US-05 to US-08; NFR-4; Figure 6

Planner	generate(user)	ValidPlan, Infeasible(conflicts, possibly_non_minimal), ProfileIncomplete(missing) or PlannerTimeout	US-09; US-12; US-18; NFR-3; Figures 4 and 5
Planner (Conflict explainer)	explain(snapshot, deadline)	List of ConstraintRef, possibly_non_minimal	US-12; Figure 7. deadline = request start + 28 s
Plan validator	validate(entries, snapshot)	List of Violation, including pool-allocation checks; empty means valid.	Planner after every solve; Saved plans on save; CSV round-trip test (US-13 AC2); 50-scenario suite (NFR-4)
Saved plans	save(user, name, entries, versions)	Plan, or PlanInvalid (400), or StalePlan (409)	US-10
Saved plans	list_plans(user) get_plan(user, id) delete_plan(user, id)	Plans with is_current; delete returns nothing	US-10; US-11; US-03 AC2
Saved plans	export_csv(user, id)	CSV text, or PlanOutOfDate (409)	US-13

6 Key Behaviors

Three flows carry most of the risk in the MVP, so each gets a behavioral diagram. **Plan generation** crosses five modules and has four outcomes. **Catalog import** is the only way data enters the catalog, and it must be all-or-nothing. **Conflict explanation** is the algorithm behind US-12. Plan is the only entity with a lifecycle, so it gets the one state machine. Error and permission paths are drawn as `alt` fragments, and the table under each figure traces every path to a Sprint 1 acceptance criterion.

6.1 Plan generation (Figures 4 and 5)

Figure 4 shows how a Generate request is authenticated, how the snapshot is read, and how an incomplete profile is rejected. Its `ref` fragment hands over to Figure 5, which shows the three solver outcomes. The snapshot is read in one REPEATABLE READ transaction, so an import committed mid-read cannot mix two catalog versions. The versions read are returned with the plan and checked again on save (Figure 8).

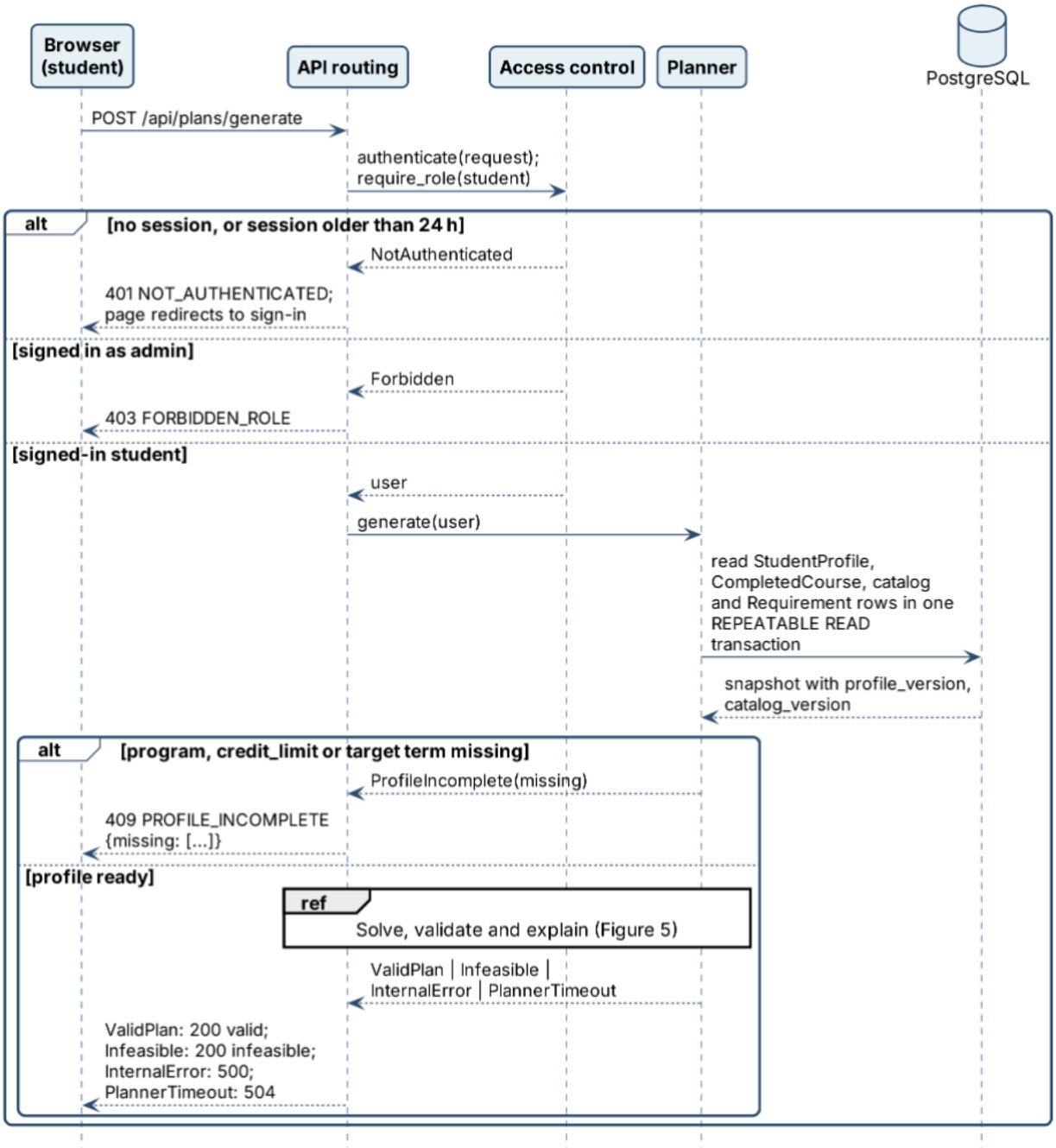


Figure 4. Plan generation, part 1: authentication, snapshot and profile check (UML sequence diagram).

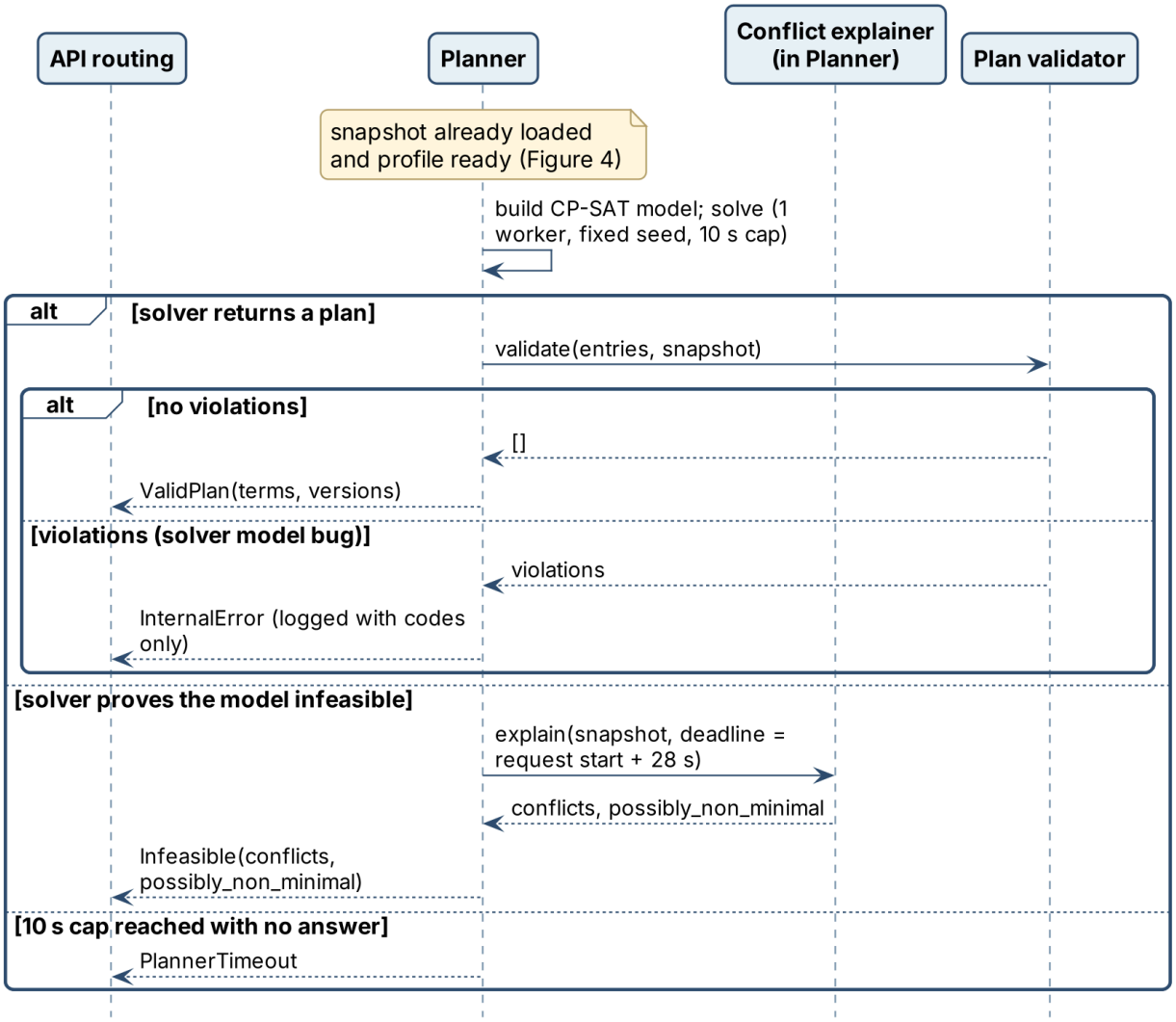


Figure 5. Plan generation, part 2: solve, validate and explain (UML sequence diagram).

Path in Figures 4 and 5	Response	Acceptance criteria
No session, or session expired	401; page redirects to sign-in	US-01 AC4; NFR-2
Signed in as admin	403	NFR-1
Program, credit limit or target term missing	409 PROFILE_INCOMPLETE listing the missing fields	US-09 AC3
Plan found and validator passes	200 valid plan; each course shows the pool it counts toward	US-09 AC1, AC2, AC4; US-18 AC1, AC2
Validator rejects the solver's plan	500; the plan is never shown	NFR-4
Model infeasible	200 infeasible with the conflict set	US-09 AC5; US-12 AC1; US-18 AC3
10-second cap reached with no answer	504 PLANNER_TIMEOUT	NFR-3

6.2 Catalog import (Figure 6)

The whole import runs in one database transaction that first locks the CatalogState row. Two admins uploading at once are therefore processed one after the other, and every check sees the same catalog. All four integrity rules from Section 4.5 run before anything is written. A rule failure writes nothing, so the rejected CatalogImport row is inserted and committed in the same transaction. The writes run inside a savepoint (a nested transaction.atomic); a database error rolls back to it, and the import is still recorded as rejected. Requirement imports follow the same flow with the US-08 checks in place of rules 1 to 4.

Cycles with OR groups. Rule 3 only follows prerequisite edges from single-course clauses (mandatory edges). A cycle through an OR alternative, such as A requires (B or C) and B requires A, is accepted, because A can still be taken after C; the planner simply never uses the blocked alternative. Rule 4 merges each corequisite pair into one node before the cycle search. That is how A → B → C ⇄ A (US-07 AC4) becomes a detectable cycle, while a plain corequisite pair (US-07 AC3) does not.

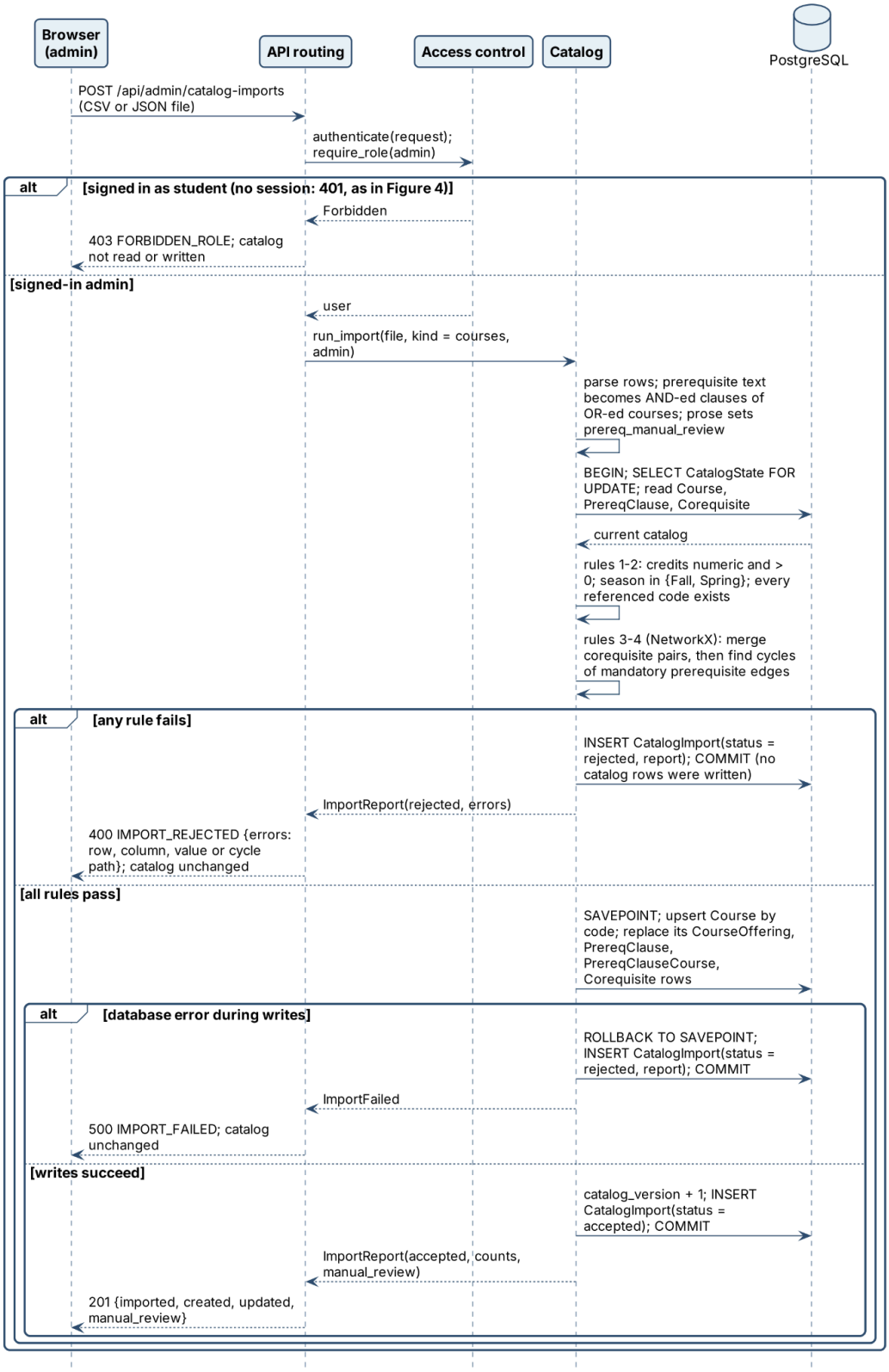


Figure 6. Catalog import with all-or-nothing validation (UML sequence diagram).

Path in Figure 6	Response	Acceptance criteria
Signed in as student	403; catalog not read or written	US-05 AC4; NFR-1
Missing or non-numeric credits	400 naming the row and column	US-05 AC2
Invalid term value	400 naming the row and value	US-05 AC5
Unknown course code in a prerequisite	400 naming the code	US-06 AC4
Prerequisite cycle	400 with the path "A → B → C → A"	US-07 AC1
Prerequisite chain contradicts a corequisite	400 with the loop "A → B → C ⇄ A"	US-07 AC4
Any rejection	Catalog identical to before	US-07 AC2; NFR-4
Valid file, existing course code	201; course updated, no duplicate	US-05 AC1, AC3
Prose prerequisite	201; course flagged for manual review	US-06 AC3
AND/OR groups and corequisite pairs	201; stored as clauses and pairs	US-06 AC1, AC2; US-07 AC3
Database error during writes	500 IMPORT_FAILED; rolled back to the savepoint, catalog unchanged	NFR-4

6.3 Conflict explanation (Figure 7)

When the generation model is infeasible, the Conflict explainer builds a second CP-SAT model with no objective. Each explainable constraint (the nine types in Section 5.4) is switched on by its own assumption literal. The solver reports a subset of assumptions that is enough for infeasibility. That core is usually not minimal, so a deletion pass removes one constraint at a time and leaves it out only if the rest is still infeasible.

Minimal means the reported constraints, together with the hard constraints, admit no valid plan, and dropping any one of them leaves a set that does. This is what US-12 AC2 and AC3 test. If a profile has two independent problems, the student sees one conflict set; after fixing it and regenerating, the next one is reported. The deletion pass needs up to one extra solve per constraint in the core, so it is bounded by one 28-second budget for the whole request; generation and explanation together stay inside the 30 seconds NFR-3 allows. Anything not tested by then stays in the set, and the result is labeled "possibly non-minimal" (US-12 AC4).

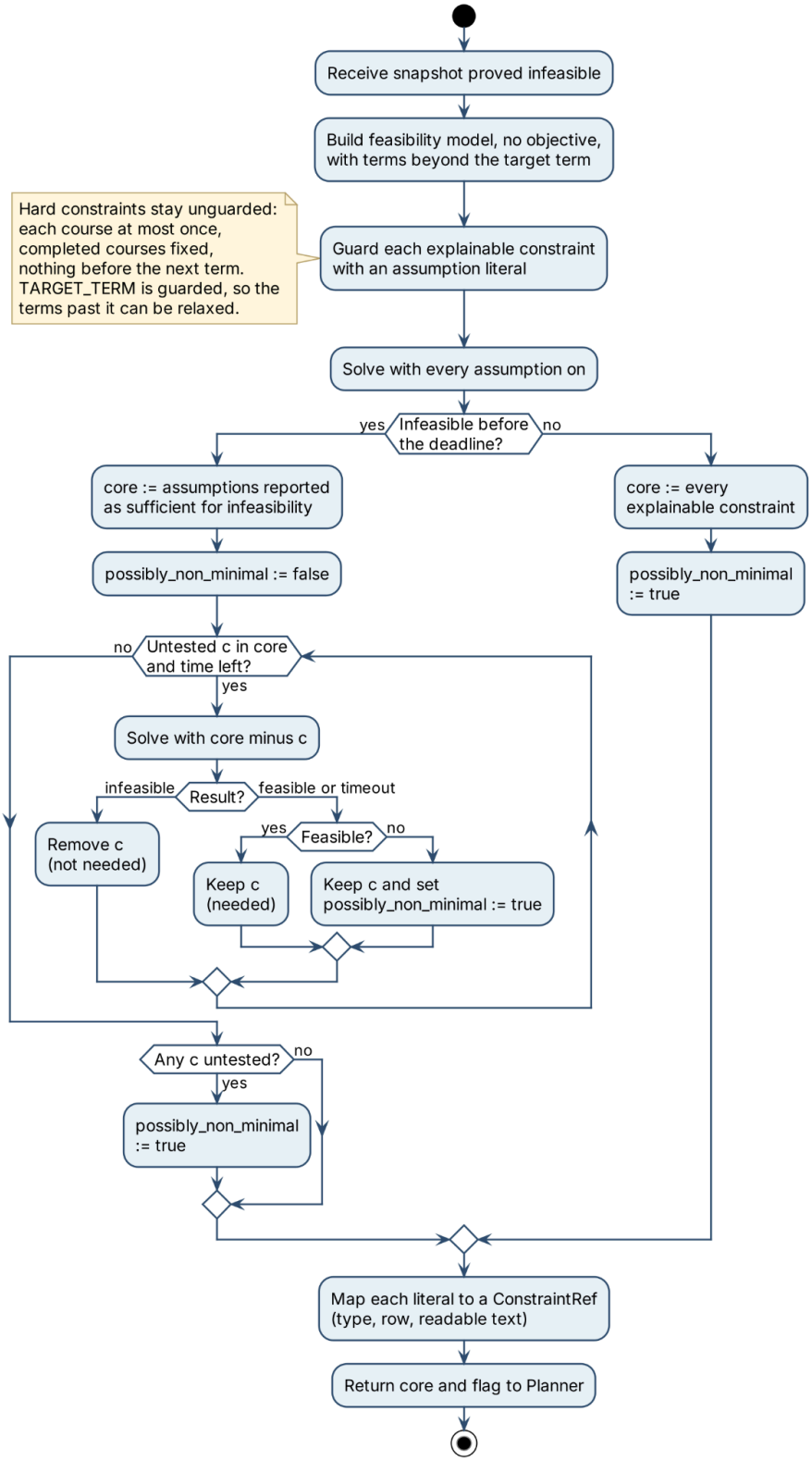


Figure 7. Conflict explanation by assumption core and deletion pass (UML activity diagram).

Acceptance criterion	How Figure 7 meets it
US-12 AC1	Only constraints left in the final core are listed, each mapped to readable text.
US-12 AC2	Every constraint kept in the core was tested: removing it made the rest feasible.
US-12 AC3	Fixture tests compare the returned core with each scenario's declared conflict.
US-12 AC4	If the deadline is reached, possibly_non_minimal is set to true.

6.4 Plan lifecycle (Figure 8)

Plan is the only entity whose behavior depends on its history. A generated plan lives only in the browser until it is saved. Saving re-runs the validator and checks that neither version number has moved. A saved plan is Current until the student's profile or the catalog changes, after which it is Out of date. That state is computed on each read rather than stored (Section 4.4), and because version numbers only increase, a plan never returns to Current. Export is allowed only from Current.

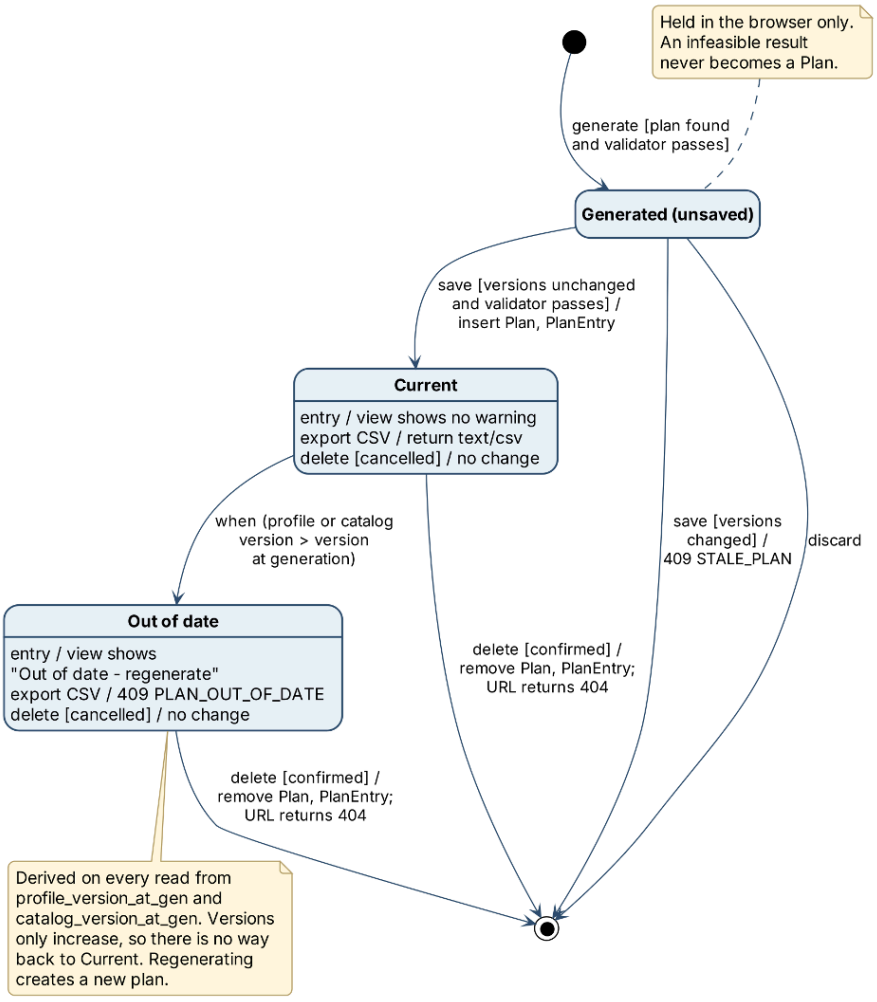


Figure 8. Plan lifecycle (UML state machine diagram).

State	Stored?	Allowed actions	Stories
Generated (unsaved)	No	Save, discard	US-09; US-10 AC1
Current	Yes	View, export CSV, delete	US-10; US-11; US-13
Out of date	Yes (state derived on read)	View with warning, delete; export refused with 409	US-03 AC2; US-11; US-13

CatalogImport was considered for a state machine, but it has only two outcomes, both decided within one request, so its behavior is shown in Figure 6 instead.

7 Design Decisions

ADR-01 React frontend with Django backend and sessions

Owner: Han Yang
Context:
The Project Ideas proposal specified React with TypeScript, FastAPI, and JWT. The team later chose to keep React and use Django for the API and login sessions.
Options:
(A) React + FastAPI + JWT follows the proposal but requires token issuance, expiry, and client-side handling. (B) React + Django + sessions uses Django's built-in login and session support while keeping the React UI.
Decision:
Use option B. React renders the student and admin interface; Django provides the JSON API and session-based login; PostgreSQL stores application data.
Consequences: The React build and Django API must be deployed together, and requests that change data need CSRF protection.

ADR-02 Transactional Imports for Degree Requirements

Owner: Hanyue Liao

Context:

DegreePath allows catalog administrators to import degree requirements, including required courses, course selection pools, and credit thresholds (US-08). The system must reject invalid requirements, such as a pool requiring students to choose more courses than the pool contains or a requirement referencing a course that does not exist in the catalog. Failed imports must not leave partially updated degree requirement data in the database.

Options:

- **(A) Incremental updates without transactions**: Write each requirement to the database as it is processed. This is simple to implement but may leave partially updated data if an error occurs.
- **(B)Transactional all-or-nothing import**: Apply all changes within a database transaction. Either the entire import succeeds or the changes are rolled back.
- **(C) Staging tables**: Import data into staging tables, validate it, and then apply valid data to the main tables. This provides additional isolation but requires more implementation complexity.

Decision:

Choose Option B: Transactional all-or-nothing import. The system validates imported requirements, including course references and pool constraints, and applies changes within a database transaction. If the import fails, the transaction is rolled back so that no partial updates are applied.

Consequences:

- Prevents partially applied degree requirement imports and helps maintain database consistency.
- Supports US-08 AC2 and AC3 by rejecting invalid course pools and references to nonexistent courses.
- Aligns degree requirement imports with the transactional approach used for catalog imports.
- Requires additional validation and transaction-handling logic; large imports may also increase transaction duration and resource usage.

ADR-03 Constraint solver for plan generation and conflict explanation

Owner: Achyuthan Sivasankar

Context. The planner must satisfy, at the same time, AND/OR prerequisite clauses, corequisites, term offerings, a per-term credit cap, a graduation window, required courses, choose-N pools and credit thresholds (US-09, US-18). When no plan exists, it must name the smallest set of conflicting constraints (US-12). Generation must finish within 10 seconds at the 95th percentile and explanation within 30 seconds (NFR-3), and the same inputs must give the same plan (US-09 AC4).

Options.

- **(A) Greedy topological scheduling with NetworkX.** Order courses by prerequisite depth and fill each term up to the credit cap. Fast and easy to follow, but pools and thresholds need a separate selection step, an early greedy choice can make a feasible profile look infeasible, and it cannot say which constraints conflict.
- **(B) Integer programming with PuLP and the CBC solver.** The same 0/1 model in linear form. Mature and free, but OR-clauses and implications must be linearized by hand, and conflict sets need extra machinery (elastic constraints or an external infeasibility tool).

- **(C) Google OR-Tools CP-SAT.** One 0/1 variable per course and term. Implications, OR-clauses and sums are stated directly, and assumption literals return a core of conflicting constraints that a deletion pass shrinks to a minimal set.

Decision. Option C. Generation and explanation share one encoding but run as two models. The generation model minimizes total scheduled credits, then prefers earlier terms. The explanation model has no objective and guards each explainable constraint with an assumption literal (Figure 7). Both run with one search worker and a fixed random seed.

Consequences.

- **Determinism costs speed.** A single worker cannot use parallel search. To keep a run that is cut off by the cap repeatable, the cap uses the solver's deterministic time limit, with the 10-second wall-clock limit as a backstop. NFR-3 is checked by 20 timed runs on the pilot major in CI.
- **Minimality costs extra solves.** Up to one re-solve per constraint in the core, bounded by the 28-second request budget, after which the result is labeled "possibly non-minimal".
- **Modeling risk.** The team must learn constraint modeling, and a modeling bug could produce a wrong plan. Mitigation: the Plan validator is independent of the solver and checks every generated plan, so bugs surface as validator failures in the 50-scenario suite (NFR-4) rather than on screen.
- **Deployment size.** The OR-Tools package increases the deployed image and memory use on the free-tier host. This will be measured on the first deployment.
- **NetworkX stays,** but only for import-time cycle checks (Figure 6).

ADR-04 Conjunctive Normal Form (CNF) Tables for Course Prerequisites

- **Owner:** Omkar Gadute
- **Context:** Course prerequisites in DegreePath include simple single-course rules, AND/OR combinations (US-06 AC1), corequisites (US-06 AC2), and unparseable prose requiring manual review (US-06 AC3). The database schema must store these rules so that the Catalog module can validate course references and detect cycles at import time (US-07), and the CP-SAT solver can translate them directly into boolean constraints (US-09).
- **Options:**
    - **(A) Raw expression string or JSON column on Course:** Store prerequisite logic as a raw string or JSON AST inside the Course table. Simple schema, but loses relational foreign-key integrity to required courses and requires re-parsing on every read.
    - **(B) Normalized CNF tables (PrereqClause, PrereqClauseCourse, Corequisite):** Normalize prerequisites into Conjunctive Normal Form (AND of OR-ed clauses) using join tables with foreign keys to Course, while keeping a boolean prereq_manual_review flag and prereq_raw_text on Course for prose rules.

- **Decision:** Choose **Option B** (PrereqClause, PrereqClauseCourse, and a canonical Corequisite table with CHECK course_id < coreq_course_id).
- **Consequences:**
    - Enforces referential integrity at the database level via foreign keys to Course (US-06 AC4).
    - Maps 1-to-1 to CP-SAT OR-clauses and NetworkX graph edges without runtime string parsing.
    - Keeps corequisites symmetric and deduplicated via the course_id < coreq_course_id constraint.
    - Requires decomposing nested boolean expressions into CNF during catalog import.

8 Traceability

8.1 Legend

Name	Type	Responsibility
Web UI	Component	React/TS pages, route guard (redirect on 401), sends `X-CSRFToken` header on unsafe requests
API Routers	Component	Django URL routes and DRF views under `/api/`, request validation via serializers
AuthService / SessionService	Component	Register, login, logout, argon2id hashing (Django `Argon2PasswordHasher`), server-side DB sessions
AccessControl	Component	`authenticate, require_role, require_owner`, 401 authentication class
ProfileService	Component	Profile read and update, `profile_version` and `catalog_version`.
CatalogService	Component	Course search, prerequisite rendering
ImportService	Component	Parse, validate, PrereqParser, CycleDetector, transactional apply
RequirementsService	Component	Degree-rule import and validation, summary
PlanService	Component	Orchestrate generate, save, list, open, delete, staleness

PlanningEngine	Component	GraphBuilder, CP-SAT planner, ConflictExplainer (pure Python, no Django imports)
PlanValidator	Component	Independent plan checker
ExportService	Component	CSV export
Repositories	Component	Thin query layer over Django ORM models (User, Profile, Catalog, Requirement, Plan) on PostgreSQL, schema via Django migrations
Logging filter	Component	Redaction and allow-listed log fields (Django `LOGGING` filter). Cookie and CSRF headers are never logged
API-1…14	Interface	REST endpoints (below)
IF-1	Interface	`PlanningEngine.solve(snapshot) → PlanResult , explain(snapshot, deadline)`
IF-2	Interface	`PlanValidator.validate(entries, snapshot) → [Violation] , run_import(file, kind, admin)`
IF-3	Interface	`ImportService.validate_and_apply(file) → ImportReport`
IF-4	Interface	Import file schema (CSV/JSON)

Endpoints (all under `/api/`)
API-1 POST `/auth/register` · API-2 POST `/auth/login` · API-3 GET `/profiles/me` · API-4 PUT `/profiles/{id}` · API-5 GET `/catalog/courses?q={text}` · API-6 POST `/admin/catalog-imports` · API-7 POST `/admin/requirement-imports` · API-8 POST `/plans/generate` · API-9 POST `/api/plans` · API-10 GET `/api/plans`, GET `/api/plans/{id}` · API-11 DELETE `/api/plans/{id}` · API-12 GET `/api/plans/{id}/export.csv` · **API-13 POST `/api/auth/logout`** · **API-14 GET `/auth/csrf`** (sets the `csrftoken` cookie, public)

8.2 MVP stories → design

Story	Components	Interfaces	Diagrams

US-01 Register / sign in	AuthService, SessionService, AccessControl, Web UI route guard, ProfileService (empty profile on register), Repositories	API-1, API-2, API-13, API-14	D1, D3
US-02 Enter profile	ProfileService, CatalogService (course-code check), Repositories, Web UI	API-3, API-4, API-15	D1, D2, D3
US-03 Update profile	ProfileService, AccessControl (ownership), PlanService (stale flag: stored profile_version_at_gen / catalog_version_at_gen vs current)	API-4, API-10	D2, D3, D6
US-04 Catalog search	CatalogService (renders PrereqExpr), AccessControl, Repositories	API-5	D1, D2, D3
US-05 Import courses	ImportService, AccessControl (`require_admin`), Repositories	API-6, IF-3, IF-4	D3, D4
US-06 Import AND/OR prereqs and coreqs	ImportService (PrereqParser), Repositories	API-6, IF-3, IF-4	D2, D4
US-07 Reject cycles	ImportService (CycleDetector on term-ordering graph), Repositories (single `transaction.atomic`)	API-6, IF-3	D2, D4
US-08 Degree requirements	RequirementsService, AccessControl, Repositories	API-7, IF-4, API-15	D2, D3, D4
US-09 Generate plan	PlanService, PlanningEngine, PlanValidator, Repositories	API-8, IF-1, IF-2	D2, D5
US-10 Save and reopen	PlanService, PlanValidator (re-validate on save), AccessControl, Repositories	API-9, API-10, IF-2	D3, D6
US-11 Delete plan	PlanService, AccessControl, Repositories	API-11	D6
US-12 Conflict explanation	PlanningEngine (ConflictExplainer), PlanService	API-8, IF-1	D2, D5
US-13 CSV export	ExportService, PlanValidator (round trip), AccessControl, PlanService	API-12, IF-2	D6

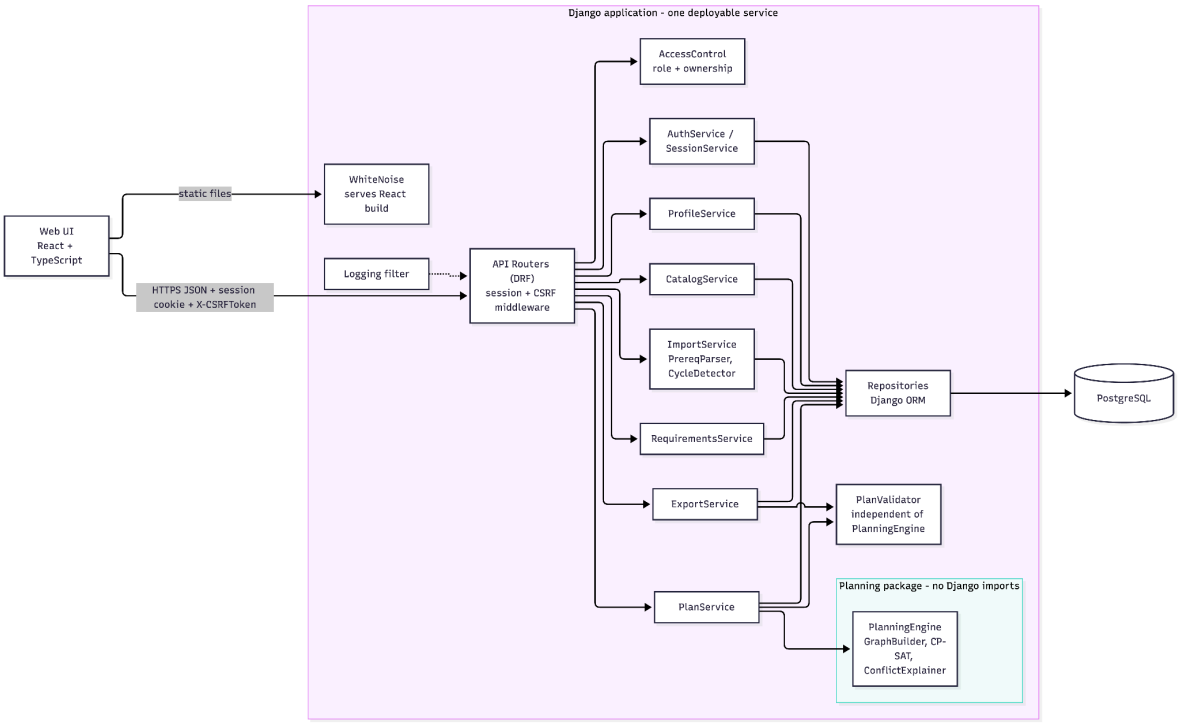
US-18 Major rules satisfied	PlanningEngine (required, pool and threshold constraints, pool allocation), PlanValidator, RequirementsService	API-8, IF-1, IF-2	D2, D3, D5

8.3 NFRs → mechanism and verification

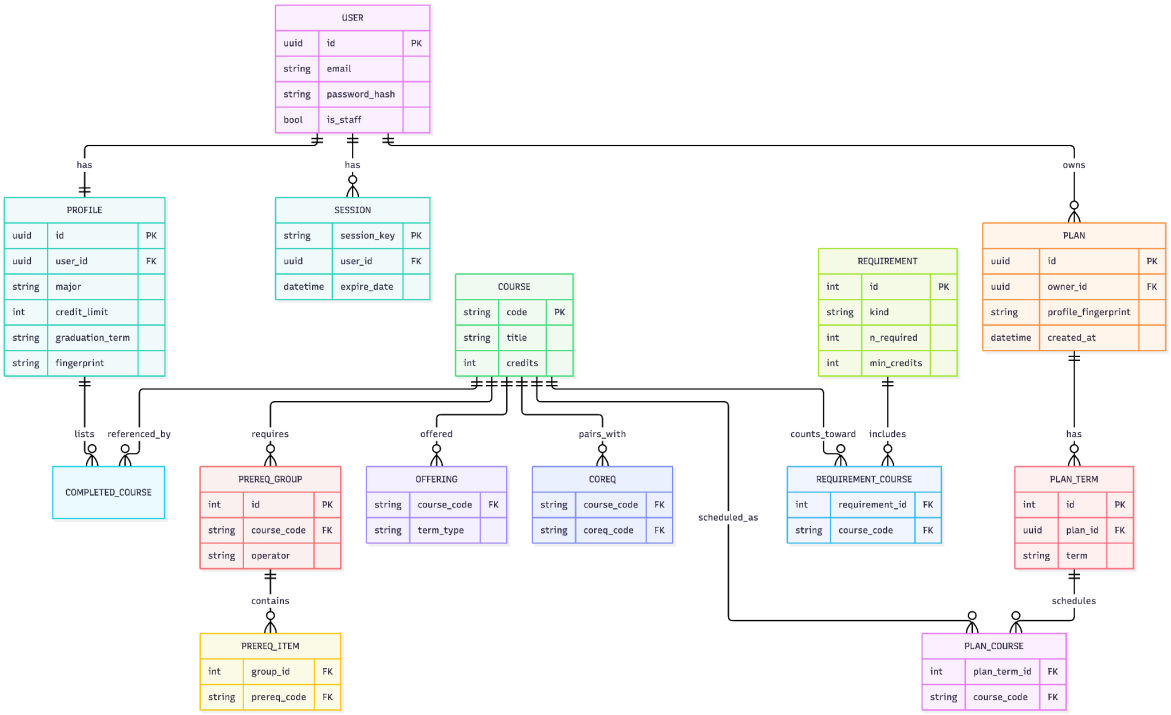
NFR	Design mechanism	Realized in	Verification
NFR-1 Permissions	Centralized permission classes and helpers (`require_role` using `User.role`, `owned_or_error`). Load by id, then `require_owner`. Random UUID IDs (DR-4). Custom DRF authentication class returns **401** for anonymous requests (DRF's stock `SessionAuthentication` returns 403). The Django admin site is limited to superusers	AccessControl, Repositories, ADR-03, Consequences	Parametrized API test per endpoint. Anonymous gets 401, student B on student A's profile or plan gets 403, and a student on API-6/7 gets 403. A route-table test fails CI if any non-public route (public: API-1, API-2, API-14) lacks a permission class
NFR-2 Security	argon2id salted hashes (`Argon2PasswordHasher` first in `PASSWORD_HASHERS`). DB-backed sessions with `SESSION_COOKIE_AGE` = 24 h. Cookie flags `HttpOnly`, `Secure`, `SameSite=Lax`. CSRF middleware enforced on all unsafe methods. Logout deletes the session row. `SECRET_KEY` from environment	AuthService, SessionService	DB-inspection test confirms every `password_hash` starts with `argon2$` and no seeded plaintext appears. Expired-session test with an injected clock returns 401. Cookie-flag test checks the `Set-Cookie` attributes. A POST without a CSRF token returns 403. After logout, replaying the old cookie returns 401

NFR-3 Performance	Single-worker CP-SAT with a deterministic time limit (DR-6), search space limited to the next term through the target term, completed courses removed, constraints grouped before minimization, 30 s explanation budget with a "possibly non-minimal" label. Gunicorn runs several workers and its timeout is above the solver budget	PlanningEngine, ADR-02, D7	A script runs 20 plan generations on the pilot-major fixture and asserts p95 ≤ 10 s, run in CI and once against the deployed demo. A forced-timeout fixture must return the label
NFR-4 Data integrity	In-memory validation plus a single-transaction import with `transaction.atomic` (DR-5), independent validator (DR-3), greedy baseline as cross-check	ImportService, PlanValidator, Section 5.1, ADR-02	An invalid-file test compares row counts before and after. A suite of at least 50 scenarios (the 15-20 infeasible fixtures plus generated feasible cases) requires 100% of plans to pass the validator
NFR-5 Privacy	Logging filter with an allow-list of fields. Request bodies for /auth, /profiles and /plans, the `Cookie`/`Set-Cookie` headers and the CSRF token are never logged. No profile data in error messages. `DEBUG=False` in deployment	Logging filter	A log-scan test runs a sample session with a canary password, canary course codes and the real session ID, then asserts none appear in captured logs
NFR-6 Persistence	All state, **including sessions**, in a non-expiring managed PostgreSQL. Stateless app container. Fixed `SECRET_KEY` in config. Migrations run on deploy. Scheduled `pg_dump`	Repositories, SessionService, D7	A restart-and-compare script snapshots profile and plans via the API, restarts the container, diffs the results, and confirms the existing session cookie is still accepted

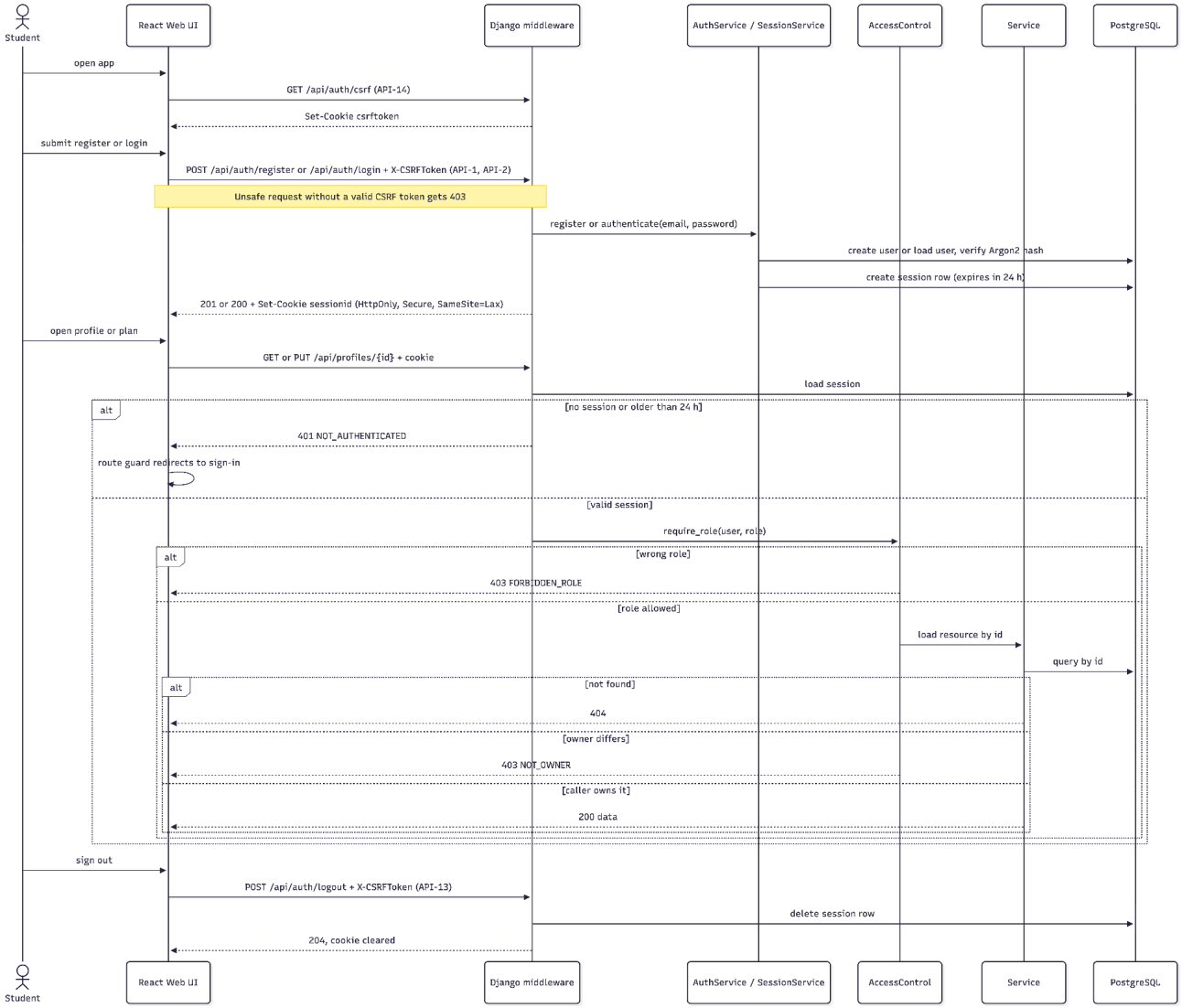
D1: Architecture



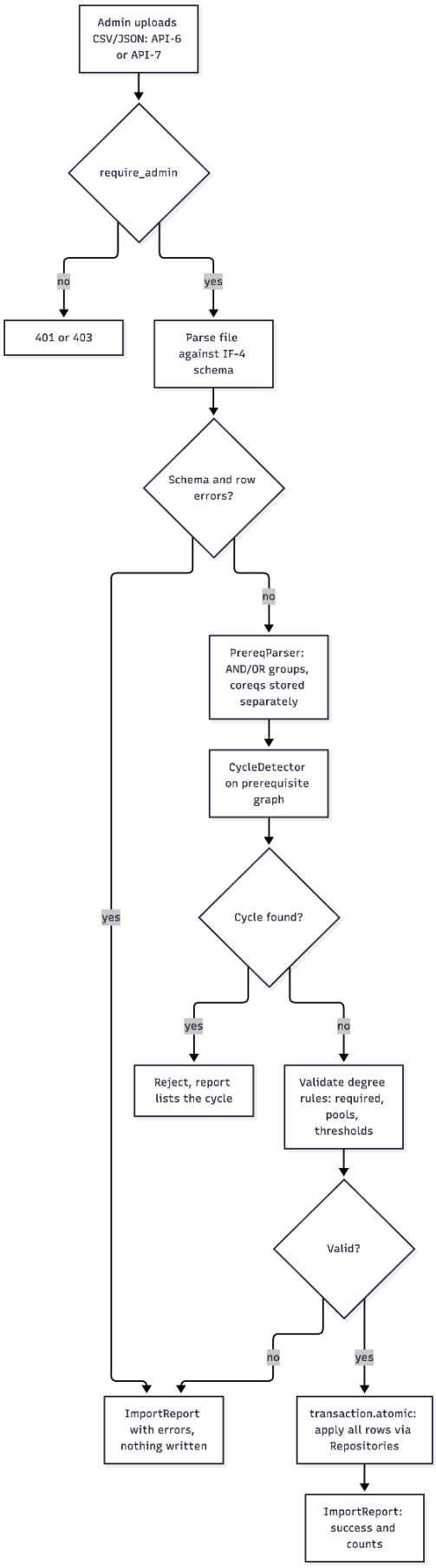
D2: Data model



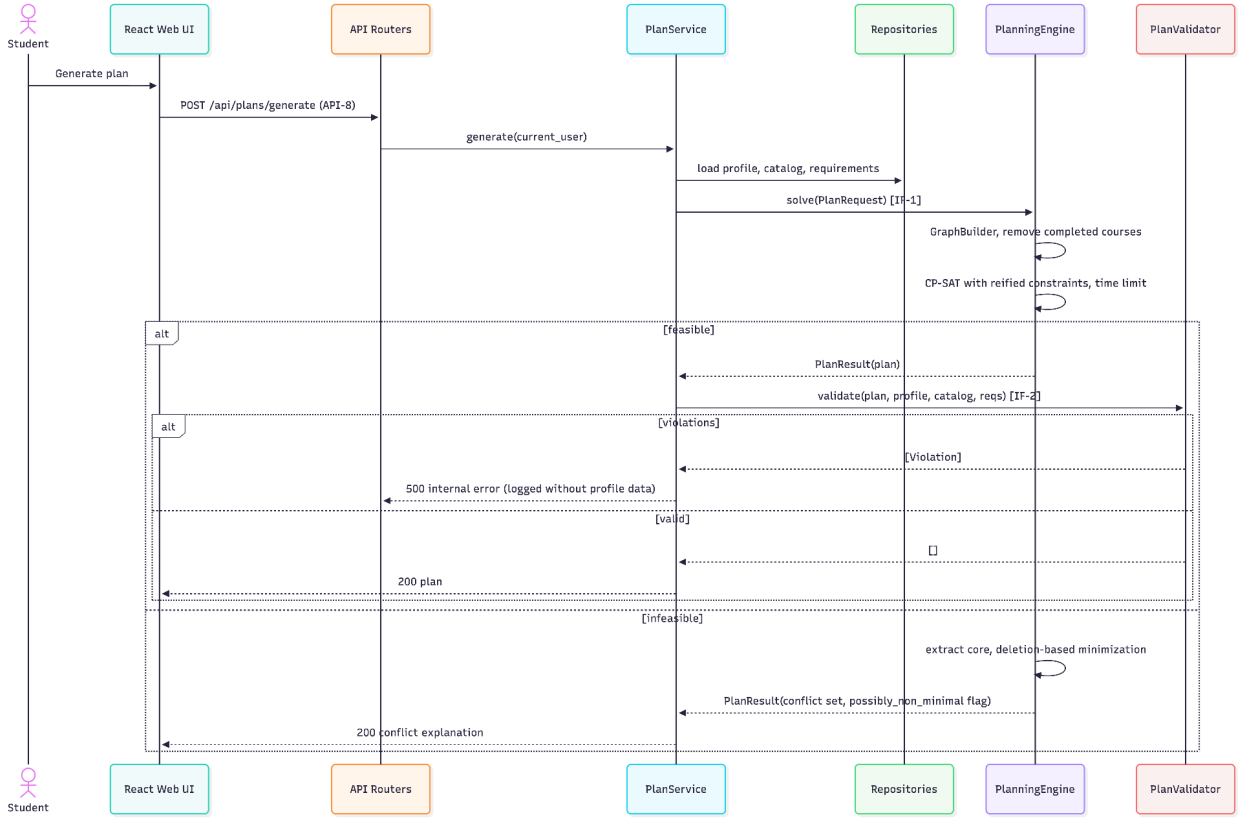
D3: Authentication and access control



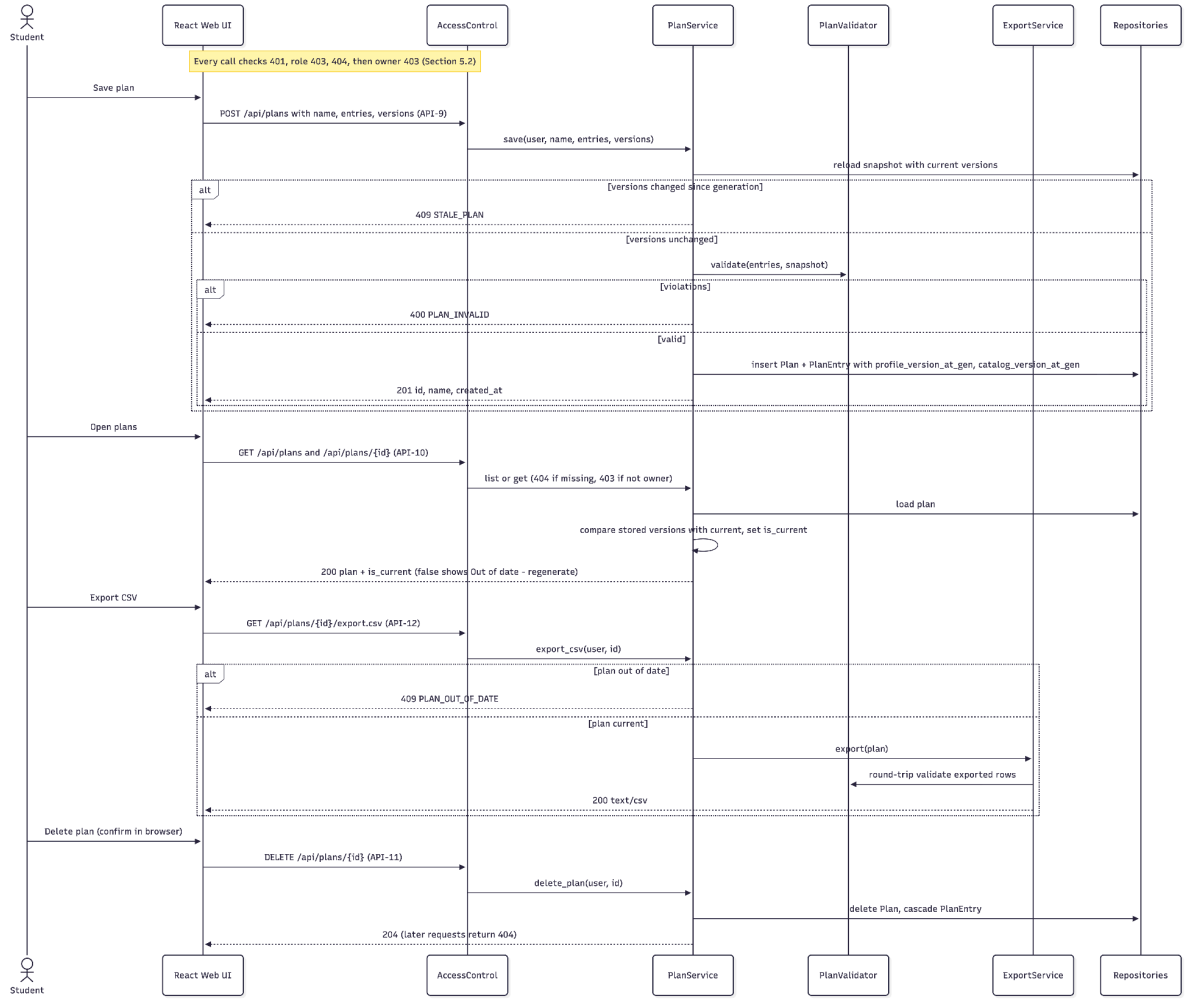
D4: Catalog / requirements import


Admin uploads CSV/JSON: API-6 or API-7
require_admin
no
yes
401 or 403
Parse file against IF-4 schema
Schema and row errors?
yes
no
PrereqParser: AND/OR groups, coreqs stored separately
CycleDetector on prerequisite graph
Cycle found?
yes
no
Reject, report lists the cycle
Validate degree rules: required, pools, thresholds
Valid?
no
yes
ImportReport with errors, nothing written
transaction.atomic: apply all rows via Repositories
ImportReport: success and counts

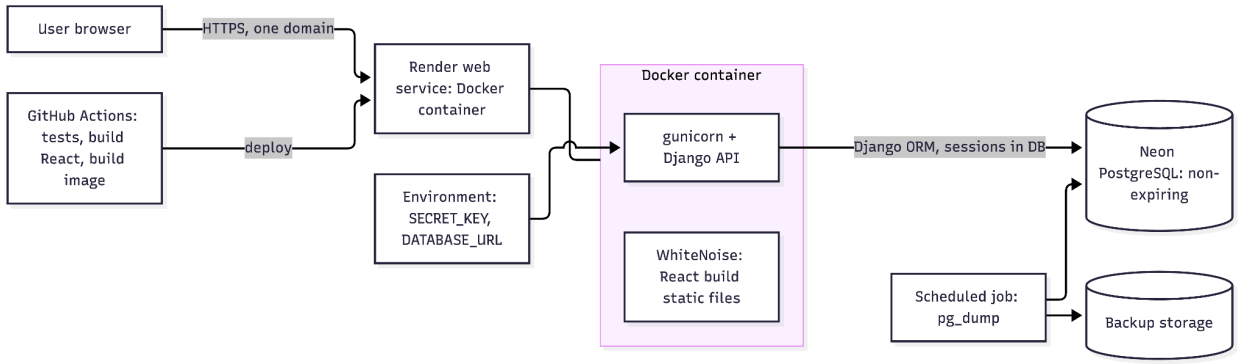
D5: Plan generation



D6: Plan lifecycle (save, open, delete, export, staleness)



D7: Deployment



9 Sprint 1 Revisions

Fix typos.

10 Design Discussion Log

This log records the design questions the team settled between 12 and 20 October while drafting and reviewing this document: who took part, the alternatives discussed, the outcome and where it is reflected. DR-1 to DR-6 were decided while drafting; the ADRs in Section 7 and the sections listed hold the full reasoning, and DR-3 to DR-6 are the records Section 8.3 cites. DR-7 onward came from review comments on this document. Status is Accepted (decided while drafting), Resolved (comment settled and the text updated) or Open (agreed or proposed; text not yet updated).

ID and date	Participants	Question and alternatives discussed	Decision and rationale	Status and where
Decided while drafting, 12–19 October (DR-3 to DR-6 are the records cited in Section 8.3)				
DR-1 19 Oct	Maitri Savaliya Achyuthan Sivasankar	Keep the Project Ideas stack (React, FastAPI, JWT) or change the backend? **(A)** React + FastAPI + JWT, as proposed in Project Ideas: the team builds token issuance, expiry and client-side handling. **(B)** React + Django + server sessions: built-in login, sessions and CSRF protection.	**B.** React for the frontend; Django for the API and session-based login. Django's built-in authentication and sessions reduce the login code the team needs to maintain. Cost: the React build and the API are deployed together, and every request that changes data needs CSRF protection.	Accepted ADR-01; Sections 2.1, 2.3, 5.1
DR-2 12 Oct	Omkar Gadute	How should AND/OR prerequisites, corequisites and prose rules be stored? **(A)** Expression string or JSON tree on Course: simple, but no foreign keys and re-parsed on every read. **(B)** Normalized CNF tables (PrereqClause, PrereqClauseCourse, Corequisite) plus a manual-review flag and raw text on Course.	**B.** Foreign keys reject unknown codes at import (US-06 AC4). Each clause maps directly to a CP-SAT OR-clause and, when it lists one course, to a NetworkX edge for cycle checks. Cost: nested expressions are converted to CNF during import.	Accepted ADR-04; Sections 4.2, 4.5
DR-3 13 Oct	Achyuthan Sivasankar	How is a bug in the CP-SAT model caught before it shows a wrong plan? **(A)** Rely on the solver model and its unit tests. **(B)** A separate Plan validator that never calls the solver or	**B.** The validator runs after every solve and again on save. A modeling bug shows up as a validator failure (500, plan never shown) in the 50-scenario suite rather than as a wrong plan on screen, and a plan edited in the browser cannot be saved (NFR-4).	Accepted ADR-03 (Consequences) ; Sections 5.5, 6.1; cited by 8.3 NFR-4

		the database and checks every plan.		
DR-4 14 Oct	Team	Which IDs do StudentProfile and Plan use, and which check runs first, existence or ownership? **(A)** Sequential integer IDs: the Django default, but they can be found by counting. **(B)** Random UUIDs.	**B,** with existence checked before ownership (404, then 403). A student can tell whether a guessed plan ID exists; the team accepts this because US-10 AC3 and US-11 AC3 require 403 for another student's plan, and UUIDs make guessing impractical.	Accepted Sections 4.2, 5.1, 5.2; cited by 8.3 NFR-1
DR-5 15 Oct	Hanyue Liao	How do catalog and requirement imports avoid leaving partial data? **(A)** Write each row as it is processed. **(B)** One database transaction per import. **(C)** Staging tables, validated and then copied.	**B.** All integrity rules (Section 4.5) run before any write. The transaction first locks the CatalogState row, so two concurrent imports run one after the other. Writes run in a savepoint, so a database error still records the import as rejected. C adds isolation the MVP does not need, at more implementation cost.	Accepted ADR-02; Sections 4.5, 6.2; cited by 8.3 NFR-4
DR-6 16 Oct	Achyuthan Sivasankar	How does the planner return the same plan for the same inputs (US-09 AC4) and still respect the 10-second cap (NFR-3)? **(A)** Parallel multi-worker search: faster, but results can vary between runs. **(B)** One search worker, a fixed random seed and a deterministic time limit.	**B.** With a deterministic limit, a run that is cut off stops at the same point every time; the 10-second wall-clock limit stays as a backstop. Cost: no parallel speed-up, so NFR-3 is checked by 20 timed runs on the pilot major in CI.	Accepted ADR-03; cited by 8.3 NFR-3
Decided in review comments on this document, 19–20 October				
DR-7 19 Oct	Raised: Omkar Gadute Resolved: Achyuthan Sivasankar, 20 Oct	Should the save request carry the pool allocation (PlanEntry.counted_for_requirement_id, US-18 AC2)? **(A)** Client sends counted_for per entry; the Plan validator verifies it. **(B)** Minimal payload; the validator recomputes the allocation on save.	**A.** Generate already returns counted_for, so the student sees the allocation that gets saved. Recomputing on save could pick a different valid allocation than the one shown, breaking US-18 AC2. Checking a given allocation is cheap and complete, so an edited request still cannot save an invalid one.	Resolved Sections 4.2 (PlanEntry), 5.3 (POST /api/plans), 5.5 (Plan validator)
DR-8 19 Oct	Raised: Omkar Gadute Assigned: Achyuthan Sivasankar	With a 10 s cap on generate() and a fixed 25 s cap on explain(), an infeasible request could take 35 s and breach the 30 s in NFR-3. How should the budget be split? **(A)** Give explain() the time left after generation.	**A,** as one deadline for the whole request: deadline = request start + 28 s. Explanation gets whatever generation did not use, which leaves 2 s of headroom under the 30 s limit. If the deadline is reached, the conflict set is labeled "possibly non-minimal" (US-12 AC4).	Resolved Sections 5.5 (Conflict explainer), 6.3; ADR-03 (Consequences)

		(B) Lower explain() to a fixed 20 s.		
DR-9 19 Oct	Raised: Hanyue Liao	A course with a prose prerequisite (for example, instructor permission) is never scheduled unless already completed. Is that a hard or an explainable constraint, and what does the conflict set return? **(A)** Hard constraint: the course is silently unavailable, and an infeasible result gives no hint why. **(B)** Explainable constraint with its own conflict type. **(C)** Treat the prose rule as satisfied and schedule the course.	**B.** MANUAL_REVIEW is the ninth ConstraintRef type. If a requirement can only be met through such a course, generation is infeasible and the conflict set names the course, so the student knows to get permission outside DegreePath. C is ruled out by US-06 AC3. Accepted limitation: the MVP cannot record that permission was granted (out of scope since Sprint 1).	Resolved Sections 4.1, 4.4, 5.4, 6.3
DR-10 20 Oct	Raised: Maitri Savaliya Assigned: Achyuthan Sivasankar	The 8.1 legend lists API-14 GET /auth/csrf, but Section 5.3 did not. Add it to 5.3 or remove it from the legend? **(A)** Add it to Section 5.3. **(B)** Remove it from the legend.	**A.** React must fetch the CSRF token before its first request that changes data, including registration and sign-in, so the endpoint is part of the API contract. The NFR-2 CSRF test in Section 8.3 depends on it.	Resolved Section 5.3 (Accounts: GET /api/auth/csrf)
DR-11 20 Oct	Raised: Maitri Savaliya Assigned: Achyuthan Sivasankar	Sections 8.1 and 8.2 use service names (ProfileService, PlanService, PlanningEngine), while Figures 1 and 4–6 and Section 5.5 use six module names, and Section 5 says the names match everywhere. Should they match? **(A)** Rename 8.1 and 8.2 to the six module names. **(B)** Keep the service names and add a column mapping each to its module. **(C)** Leave as is.	**Proposed: B.** The services split the modules more finely (ImportService and RequirementsService both belong to Catalog; PlanService spans Planner and Saved plans), so renaming would lose detail. C leaves the statement in Section 5 false.	Open Section 8.1 legend not yet updated
DR-12 20 Oct	Raised: Hanyue Liao	Can program_id be null while a student sets up a profile? The Figure 4 table treated a missing program as PROFILE_INCOMPLETE, but Section 4.4 defined readiness by credit limit and target term only. **(A)** Require a program at registration. **(B)** Nullable until set, and part of readiness.	**B.** Registration creates an empty profile, so program_id is nullable until the student sets it, like the credit limit and target term. A profile is ready only when all three are set, because the degree requirements come from the program. Otherwise generate returns 409 PROFILE_INCOMPLETE listing the missing fields (US-09 AC3).	Resolved Sections 4.2, 4.4; Section 6.1 path table

DR-13 20 Oct	Raised: Han Yang	Section 8.2 cites API-15 for US-02 and US-08, but the legend stops at API-14 and the programs endpoints in Section 5.3 have no number. **(A)** Number the programs endpoints API-15 and add them to the legend. **(B)** Remove the API-15 references.	**A.** GET /api/programs and GET /api/programs/{id} serve the program choice in US-02 and the requirement summary in US-08. Removing the references would leave both stories without the endpoint that supports them.	Open Section 8.1 legend still ends at API-14
DR-14 20 Oct	Raised: Han Yang Replied: Maitri Savaliya	Some paths in the 8.1 endpoint list include /api/ and others do not, although the heading says all are under /api/. Use one format? **(A)** Drop /api/ everywhere, since the heading states it. **(B)** Copy the full paths from Section 5.3.	**B.** Section 5.3 stays the single source for paths. Copying from it also corrects API-3 (GET /profiles/me) and API-5 (/catalog/courses), which name paths that Section 5.3 does not define.	Open Section 8.1 endpoint list not yet updated
DR-15 20 Oct	Raised: Han Yang	Does Sprint 2 change any Sprint 1 artifact?	No user story or acceptance criterion changes. A typo in the Sprint 1 document is corrected and listed in Section 9.	Open Section 9 still reads "None."